Recovery from Temporary Gradient Sign Corruption in SGD, AdamW, signSGD, and Lion

JEGOU Joanne, CHAOUI EL FAIZ Kamil, LORTHIOIS Thybault
Optimization for Machine Learning, EPFL, Spring 2026

***Abstract*—We study how SGD with momentum, AdamW, signSGD, and Lion recover after a short burst of gradient sign corruption on CIFAR-10. During epochs 20–25 of a 60-epoch run, we flip the sign of a fraction $p \in \{5, 10, 20, 40\}\%$ of gradient coordinates, then remove the perturbation and measure recovery. We test a fresh random mask at every step and a fixed persistent mask. Lion is the only optimizer to survive p=40% in both modes; AdamW and SGD collapse to chance level and do not recover. The key difference is Lion's EMA buffer, which filters adversarial gradient biases before the sign is applied. Layer-wise experiments show that catastrophic failure requires all layers to be corrupted simultaneously; a single corrupted layer causes less than 2% accuracy drop even at p=40%.**

I. Motivation and Related Work

CNNs remain brittle under distribution shift. Hendrycks and Dietterich [1] showed that blur or brightness corruptions reliably degrade accuracy, and Hosseini et al. demonstrated that impulse noise fools Google's Cloud Vision API [2]. Bishop showed that training with Gaussian noise is equivalent to Tikhonov regularization [3], and Ranabhat et al. confirmed that regularization improves corruption robustness [4]. Direct gradient corruption bypasses nonlinear activations and hits each optimizer's update rule immediately—a cleaner perturbation than input noise.

We compare four optimizers: AdamW with decoupled weight decay [5]; signSGD, a compression scheme keeping only the coordinate-wise sign [6] (naive sign compression can lose direction information [7]); Lion, found by program search, updating via the sign of a momentum estimate [8]; and SGD with momentum as baseline. Sign-based methods have been linked to robustness under heavy-tailed noise [9]; we ask whether this transfers to structured, transient sign attacks.

Prior work focuses on persistent Byzantine attacks in distributed settings [6] or steady-state gradient noise. We study something different: a *short burst* that is then cleanly removed. This regime exposes each optimizer's internal memory—how much corrupted information it retains and how fast it forgets. To our knowledge, no prior work measures recovery dynamics under temporary sign corruption across optimizers with fundamentally different update rules.

II. Method

A. Experimental Setup

We train a small CNN on CIFAR-10 [10] for 60 epochs, batch size 128, with CIFAR-10 channel statistics, random crop and horizontal flip. All runs share seed 42 and identical model initialization. Hyperparameters (Table I) are chosen by grid search on the clean (p=0) setting and held fixed for all corruption runs; the clean runs are our baselines. The burst spans epochs 20–25, after training has stabilized. The referenced CNN reports 80.17% accuracy; our baselines are lower due to the smaller network capacity we use. Chance level for a classification task with 10 labels is at 10%.

What did not work: SignSGD at its default η=10^{-2} collapses to chance level ($\approx$10%), consistent with the instability in [7]. We reduce it to η=10^{-3}, which gives a stable 40.2% baseline at the expense of a substantially higher computational time. SignSGD was excluded from the layer-wise corruption analysis due to its extreme sensitivity to the learning rate, and the resulting difference in hyperparameter settings. All other optimizer defaults transfer directly from standard CIFAR-10 practice.

B. Temporary Sign Corruption

After `loss.backward()`, a fraction p of gradient coordinates has its sign flipped:

$$g'_{t,i} = \begin{cases} -g_{t,i}, & i \in S_t, \\ g_{t,i}, & \text{otherwise}, \end{cases} \qquad |S_t| = \lfloor p\, d \rfloor. \tag{1}$$

Two masking strategies: **Random** (S_t resampled i.i.d. each step) and **Persistent** (S_t fixed for the whole burst). We also test layer-specific corruption: *stem* (`conv1`), *head* (`fc2`), and *global* (all layers).

C. Metrics

Recovery time is the first epoch after burst end satisfying $a_t \geq 0.98\, a_{\text{final}}^{\text{clean}}$; DNR (Do Not Recover) if never met. We also report the final accuracy gap and the accuracy drop at burst end. All code and data are available on Github; figures are reproduced by `generate_paper_figures.py`.

III. Experimental Results

A. Hyperparameters and Baselines

Table I lists the hyperparameters. The clean baselines at epoch 60 are: SGD 60.3%, AdamW 71.4%, signSGD 40.2%, Lion 71.1%. These serve as reference throughout the study; all accuracy gaps are relative to these values. SignSGD's lower baseline reflects its simpler update (no adaptive scaling), not its corruption behavior.

TABLE I
OPTIMIZER HYPERPARAMETERS. LEARNING RATES CHOSEN BY GRID SEARCH ON CLEAN CIFAR-10; ALL OTHER PARAMETERS FOLLOW STANDARD PRACTICE.

Optimizer	η	Momentum	Weight Decay	Note
SGD	5×10^{-2}	0.9	5×10^{-4}	–
AdamW	1×10^{-3}	0.9/0.999	10^{-2}	–
signSGD	1×10^{-3}	–	–	reduced from 10^{-2}
Lion	1×10^{-4}	0.9/0.99	10^{-2}	–

TABLE II
FINAL VALIDATION ACCURACY (%) AT EPOCH 60, RANDOM MASK. **BOLD** = BEST. † ABOVE CHANCE BUT BELOW RECOVERY THRESHOLD.

	Baseline	p=5%	p=10%	p=20%	p=40%
SGD	60.3	64.0	63.2	59.8	10.0
AdamW	71.4	70.0	70.4	70.6	10.0
signSGD	40.2	31.0	32.4	34.1	30.0†
Lion	**71.1**	**71.4**	**71.2**	**71.2**	**69.8**

B. Random Corruption

Table II and Figure 1 show the results.

At $p \leq 20\%$, all optimizers recover, but at very different speeds. SGD recovers fast at low p (2 epochs at p=5%, 8 at p=10%) but needs 34 epochs at p=20%. AdamW takes 21–28 epochs at all levels: its second moment v_t absorbs corrupted gradient squares and must be diluted by many clean steps. This asymmetry is notable—SGD drops more at burst end (11.9% at p=5%) but recovers faster than AdamW (which drops only 4% but takes 27 epochs). Lion recovers in 15–17 epochs regardless of p and stays within 0.2% of its clean baseline. SignSGD shows 0 recovery time at $p \leq 20\%$: the accuracy never fell below the 98% threshold *during the burst*, consistent with the sign-majority argument ($\geq$ 60% of signs remain correct at $p \leq 20\%$). The final accuracy (31–34%, Table II) is below the 40.2% baseline, reflecting signSGD's inherently noisy training dynamics rather than unrecovered corruption damage.

At p=40%, SGD and AdamW collapse to chance (10%) and do not recover. Lion ends at 69.8% after 33 recovery epochs—the only optimizer to survive. SignSGD reaches 30.0%, above chance but not fully recovering (DNR).

C. Persistent Corruption

At p=5% the burst is barely visible (Figure 2): all four optimizers track their clean baselines and re-converge within a few epochs. The persistent mask at low corruption adds a small fixed bias that the network absorbs without lasting damage.

At p=20% (Figure 3), SGD collapses to chance during the burst then climbs back over 34 epochs. AdamW barely dips but also takes ≈28 epochs to return—its buffered moments carry the corruption forward. Lion barely moves. SignSGD partially degrades but stays above chance.

At p=40% (Figure 4), SGD and AdamW collapse at the first burst epoch and never recover. Lion ends at 69.8%, unchanged from its random-mode result. SignSGD retains 30%. The mask structure (random vs. persistent) does not change which optimizers survive.

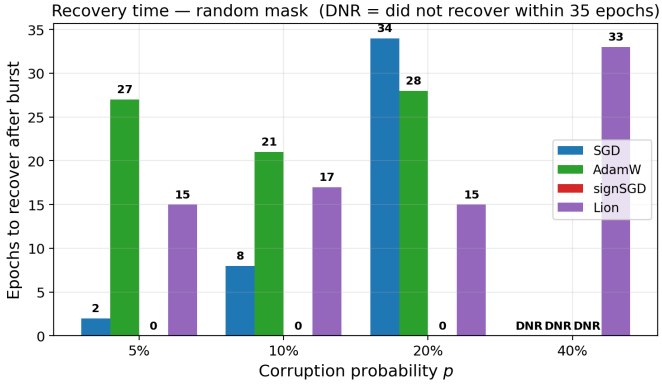


Fig. 1. Recovery time (epochs post-burst) under random mask, all layers. DNR = never recovered. "—" = accuracy never dropped below threshold. At p=40% only Lion recovers (33 epochs).

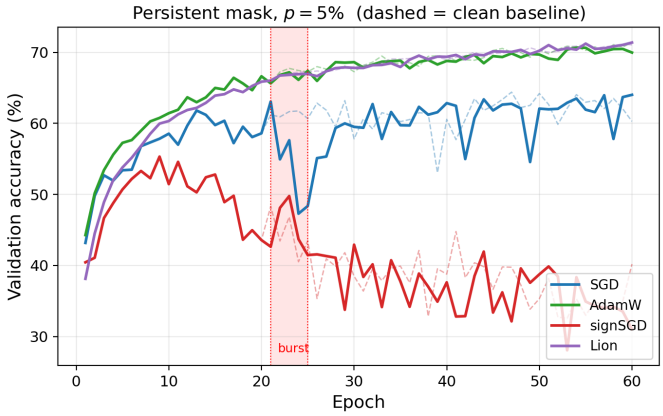


Fig. 2. Persistent mask, p=5%, all layers corrupted. All optimizers track their clean baseline; the burst has no lasting effect.

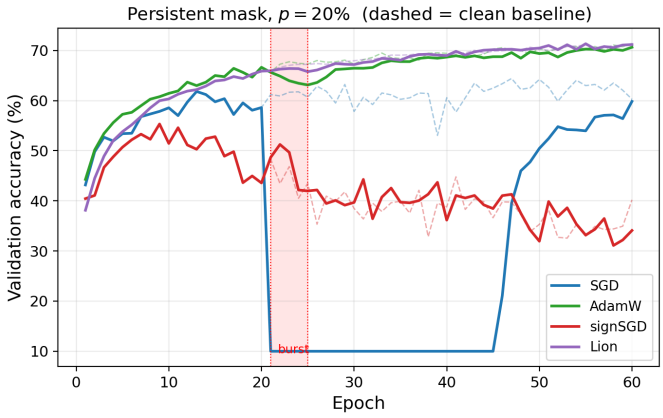


Fig. 3. Persistent mask, p=20%. SGD collapses during burst then recovers slowly (34 epochs). AdamW dips less but also recovers slowly (28 epochs). Lion is nearly unaffected.

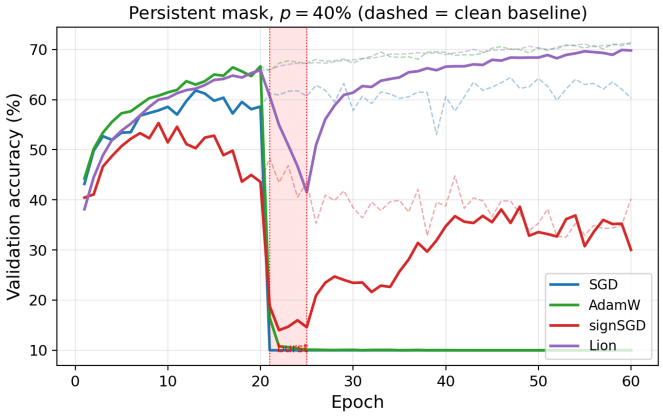


Fig. 4. Persistent mask, p=40%, all layers. SGD and AdamW collapse to chance at the first burst epoch and never recover. Lion ends at 69.8%; signSGD degrades to 30% but stays above chance.

TABLE III
ACCURACY GAP (%) AT p=40% BY CORRUPTION TARGET (RANDOM MASK). — = SIGNSGD EXCLUDED (LR SENSITIVITY IN HEAD SETTING).

Target	SGD	AdamW	Lion	signSGD
Global	50.3	61.4	1.3	10.2
Stem	2.1	1.0	0.3	7.8
Head	1.1	0.1	0.2	—

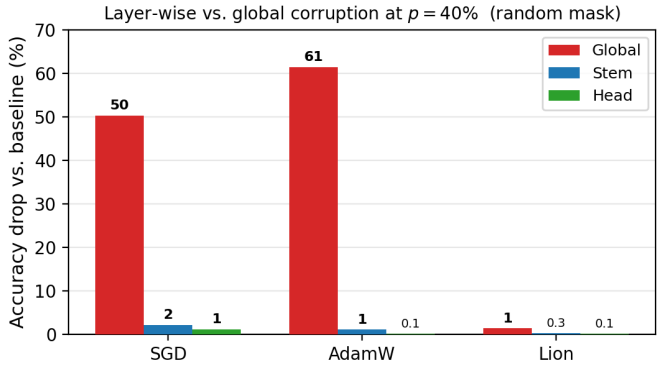


Fig. 5. Accuracy drop at p=40% (random mask) for global, stem, and head corruption. Single-layer corruption causes at most 2% degradation; global corruption destroys SGD and AdamW.

D. Layer-wise Corruption

We restrict the random mask to *stem* (`conv1`) and *head* (`fc2`) individually. Table III and Figure 5 compare the accuracy drop at p=40% across targets. Single-layer corruption causes at most 2.1% drop for SGD and below 0.3% for AdamW and Lion—versus 50% and 61% in global mode. Neither layer alone triggers collapse. When only one layer is perturbed, the rest of the network provides correct descent directions and compensates. Lion stays below 0.3% regardless of target, extending its global robustness to the single-layer setting.

E. Discussion

a) Why Lion survives.: Lion updates via sign(m_t) with $m_t = \beta_1 m_{t-1} + (1-\beta_1) g_t$, β_1=0.9. Corrupting p coordinates of g_t shifts m_t by only $(1-\beta_1)\sum_s \beta_1^{k-s}\delta_s$. After ≈1950 gradient steps (5 epochs × 391 steps/epoch at batch size 128), the pre-burst momentum dominates: flipping a coordinate in m_t requires the perturbation to outweigh ≈1950 steps of clean signal. SGD uses momentum only for acceleration, not smoothing. AdamW adapts both m_t and v_t to the corrupted signal, compounding rather than filtering the damage.

b) AdamW's slow recovery.: Corrupted gradients inflate v_t, suppressing updates at affected coordinates. Simultaneously m_t is pulled the wrong way. Both effects persist after the burst ends: v_t stays inflated until clean gradients dilute it, which is why AdamW's recovery takes three times longer than its burst-end accuracy drop would suggest.

c) SignSGD and sign majority.: Corrupting p coordinates leaves $1-p$ pointing correctly; the net signal is $1-2p$ of the clean gradient (20% at p=40%). That residual prevents divergence to chance but not accuracy loss. At $p \leq 20\%$ the residual exceeds 60%, which explains the zero recovery time.

IV. CONCLUSION

In this study, we investigated how different optimizers recover from transient gradient sign attack, at different corruption levels. We looked into different settings, with random and persistent corruption, and how targeted layers impact (or fail to impact) degradation and recovery.

The four examined optimizers respond to the same 5-epoch sign corruption burst in four distinct ways. At p=40%, SGD and AdamW collapse irreversibly; signSGD degrades but stays above chance; Lion survives both modes with a final gap under 1.5%. The deciding factor is whether the optimizer smooths gradient information before the sign: Lion's EMA buffer provides this naturally, signSGD benefits from sign majority, and SGD/AdamW have no comparable mechanism. Layer-wise experiments show that no single layer drives failure: catastrophic collapse requires simultaneous perturbation across the whole network, which suggests layer-specific defenses could limit damage in practice. Lion's implicit robustness to sign attacks in centralized training had not been documented before and may be relevant for Byzantine-resilient federated settings.

REFERENCES

[1] D. Hendrycks and T. Dietterich, "Benchmarking neural network robustness to common corruptions and perturbations," 2019. [Online]. Available: https://arxiv.org/abs/1903.12261

[2] H. Hosseini, B. Xiao, and R. Poovendran, "Google's cloud vision api is not robust to noise," IEEE, 2017. [Online]. Available: https://arxiv.org/abs/1704.05051

[3] C. Bishop, "'training with noise is equivalent to tikhonov regularization'," pp. 108–116, 1995. [Online]. Available: https://www.microsoft.com/en-us/research/wp-content/uploads/2016/02/bishop-tikhonov-nc-95.pdf

[4] R. N. Ranabhat, L. Wang, A. K. Patel, and K. santosh, "Promoting shape bias in cnns: Frequency-based and contrastive regularization for corruption robustness," 2025. [Online]. Available: https://arxiv.org/abs/2509.11355

[5] I. Loshchilov and F. Hutter, "Decoupled weight decay regularization," in *International Conference on Learning Representations*, 2019. [Online]. Available: https://arxiv.org/abs/1711.05101

[6] J. Bernstein, Y.-X. Wang, K. Azizzadenesheli, and A. Anandkumar, "signsgd: Compressed optimisation for non-convex problems," in *Proceedings of the 35th International Conference on Machine Learning*. PMLR, 2018, pp. 560–569. [Online]. Available: https://arxiv.org/abs/1802.04434

[7] S. P. Karimireddy, Q. Rebjock, S. U. Stich, and M. Jaggi, "Error feedback fixes signsgd and other gradient compression schemes," in *Proceedings of the 36th International Conference on Machine Learning*. PMLR, 2019, pp. 3252–3261. [Online]. Available: https://arxiv.org/abs/1901.09847

[8] X. Chen, C. Liang, D. Huang, E. Real, K. Wang, Y. Liu, H. Pham, X. Dong, T. Luong, C.-J. Hsieh, Y. Lu, and Q. V. Le, "Symbolic discovery of optimization algorithms," in *Advances in Neural Information Processing Systems*, 2023. [Online]. Available: https://arxiv.org/abs/2302.06675

[9] D. Yu, H. Tao, Y. Wan, L. Luo, and L. Zhang, "Sign-based optimizers are effective under heavy-tailed noise," *arXiv preprint arXiv:2602.07425*, 2026. [Online]. Available: https://arxiv.org/abs/2602.07425

[10] R. Chauhan, K. K. Ghanshala, and R. Joshi, "Convolutional neural network (cnn) for image detection and recognition," in *2018 First International Conference on Secure Cyber Computing and Communication (ICSCCC)*, 2018, pp. 278–282.